

```

1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 from time import sleep
5 from gpiozero import Servo
6 from gpiozero.pins.pigpio import PiGPIOFactory
7 from aiy.pins import PIN_A, PIN_B # Assuming AIY Voice Kit pins
8
9 # -----
10 # Servo Initialization (using pigpio factory for better precision)
11 factory = PiGPIOFactory()
12 servo_x = Servo(PIN_A, min_pulse_width=0.0005, max_pulse_width=0.0019, pin_factory=factory)
13 servo_y = Servo(PIN_B, min_pulse_width=0.0005, max_pulse_width=0.0019, pin_factory=factory)
14
15 # Gradual movement function (from your example)
16 def move_servo_gradually(servo, start_value, end_value, steps=20, delay=0.01):
17     if abs(end_value - start_value) < 0.01: # No significant change, skip
18         return
19     step_size = (end_value - start_value) / steps
20     current = start_value
21     for _ in range(steps + 1):
22         servo.value = current
23         sleep(delay)
24         current += step_size
25     servo.value = end_value # Ensure final position
26
27 # -----
28 mp_face_detection = mp.solutions.face_detection
29 mp_face_mesh = mp.solutions.face_mesh
30 mp_drawing = mp.solutions.drawing_utils
31 mp_drawing_styles = mp.solutions.drawing_styles
32
33 cap = cv2.VideoCapture(0)
34 cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640) # Standard width for consistent mapping
35 cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480) # Standard height
36
37 with mp_face_detection.FaceDetection(min_detection_confidence=0.5) as face_detection, \
38     mp_face_mesh.FaceMesh(
39         min_detection_confidence=0.5,
40         min_tracking_confidence=0.5,
41         max_num_faces=1,
42         static_image_mode=False
43 ) as face_mesh:

```

Fig. 7: Code snippet for face tracking

```

Servo_test.py | objecttrack.py | Face_Track_servo.py
1 # =====
2 # MediaPipe Object Detection
3 # - Green boxes + labels for all objects
4 # - Tracked object: red box + GREEN DOT at center + coordinates
5 # =====
6 import cv2
7 import mediapipe as mp
8 from mediapipe.tasks import python
9 from mediapipe.tasks.python import vision
10 import numpy as np
11
12 # =====
13 # CONFIGURATION
14 # =====
15 MODEL_PATH = "efficientdet_lite0.tflite" # Recommended: use efficientdet_lite0 (remove space)
16 # Download: https://storage.googleapis.com/mediapipe-models/object_detector/efficientdet_lite0/float16/1/efficientdet_lite0.tflite
17
18 BaseOptions = python.BaseOptions
19 ObjectDetector = vision.ObjectDetector
20 ObjectDetectorOptions = vision.ObjectDetectorOptions
21 VisionRunningMode = vision.RunningMode
22
23 options = ObjectDetectorOptions(
24     base_options=BaseOptions(
25         model_asset_path=MODEL_PATH,
26         delegate="CPU" # change to "GPU" if you have it working
27     ),
28     running_mode=VisionRunningMode.VIDEO,
29     max_results=15,
30     score_threshold=0.35
31 )
32
33 # =====
34 # DRAW FUNCTION - with green center dot for tracked object
35 # =====
36 def draw_detection(image, detection_result, tracked_object=None):
37     annotated_image = image.copy()
38     h, w, _ = image.shape
39
40     for detection in detection_result.detections:
41         bbox = detection.bounding_box
42         start_x = int(bbox.origin_x)
43         start_y = int(bbox.origin_y)
44         end_x = int(bbox.origin_x + bbox.width)
45         end_y = int(bbox.origin_y + bbox.height)
46
47         category = detection.categories[0]
48         label = category.category_name
49         score = category.score
50
51         is_tracked = tracked_object and label.lower() == tracked_object.lower()
52
53         # Box color: red for tracked, green for others
54         color = (0, 0, 255) if is_tracked else (0, 255, 0)
55         thickness = 4 if is_tracked else 2
56
57         # Draw bounding box

```

Fig. 8: Object tracking code and user prompt

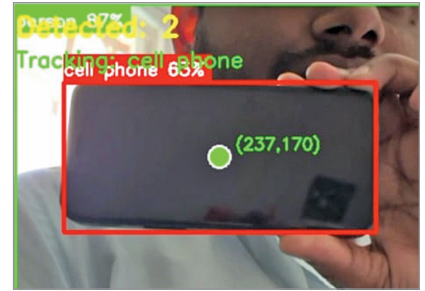


Fig. 9: Real-time object tracking output

TO DOWNLOAD SOURCE CODE AND RESOURCES



Scan this QR code or go to <https://sourcecode.electronicsforu.com> to access and download source code and other resources for this project

Fig. 3 for more details.

In the program, a prompt is provided for the user to enter the name of the object to track, such as fan, bottle, person, cell phone, or a TV remote. Fig. 8 shows the object tracking code and the serial output prompting the user to enter the object name to track.

Testing

To test, run the face-tracking code. The output will display the tracked face, and the robot will move its head and eyes to follow the face in real time.

For object tracking, run the objecttrack.py code. A window will open asking for the object name. Enter an object name such as ball, bottle, person, or mobile phone. Once entered, the camera will automatically start tracking the selected object and move accordingly, as shown in the author's prototype (see Fig. 1). Similarly, Fig. 9 shows the real-time object tracking output for a detected cell phone with a bounding box, confidence level, and tracking coordinates. **EFY**

Ashwini Kumar Sinha, an IoT and AI enthusiast, is Tech Journalist at EFY.